MongoDB + AirBnB

Stage 3 Prototype

COURSE CS 498 · Data Management in the Cloud

TEAM Abhinav Bachu · Hanshul Bahl · Alejandra Arias

REPO github.com/abachu2005/cs498-airbnb-mongodb-prototype

DATE April 29, 2026

Project recap & Stage 3 scope

Where we've been

- **Stage 1** — picked **MongoDB + InsideAirbnb** (4 cities)
- **Stage 2** — designed **4 collections, 8 indexes**, pseudocode for all 6 queries

What Stage 3 delivers

- Loader against the **full real datasets** — no row caps
- **All 6 of 6** queries implemented and benchmarked **(min for a 3-person team is 4)**
- `explain('executionStats')` captured for every query

Two ways to run it

```
# CLI evidence pipeline
npm start

# interactive demo UI
npm run web
```

+10 / +10

EXTRA-CREDIT ELIGIBILITY FROM QUERIES + DATA COVERAGE

Data model

4 collections

`listings`	per-listing metadata
`calendar`	per-day availability + price
`reviews`	one row per review
`neighborhoods`	canonical names per city

> Time-series collections (`calendar`, `reviews`) stay separate from `listings` so we never rewrite a giant array on every booking change.

8 indexes (carried from Stage 2 unchanged)

```
listings    { city, listing_id }              UNIQUE
listings    { city, neighborhood, room_type }
listings    { host_id, city }
calendar    { city, listing_id, date }
calendar    { city, date, available }
reviews     { city, listing_id, date }
reviews     { reviewer_id, listing_id, date }
neighborhoods { city, neighborhood }          UNIQUE
```

Loader pipeline + data coverage

```
InsideAirbnb URL  →  fetch  →  gunzip stream  →  csv-parse  →  batched insertMany  →  build 8 indexes
```

63,482
LISTINGS

23.17 M
CALENDAR ROWS

3.20 M
REVIEWS

481
NEIGHBORHOODS

~6.5 min
TOTAL LOAD + INDEX BUILD

4 / 4
CITIES (LA · PORTLAND · SALEM · SAN DIEGO)

The entire InsideAirbnb dataset for all four cities — not a subset. (Assignment: "use of all or nearly all the data is worth 10 percentage points.") Streaming + batched inserts keep heap flat.

Q1 · Two-night search (Portland)

demo | IXSCAN | 112 ms

Goal. Listings in Portland whose `calendar` is open for **both** nights of a 2-day window, ranked by `review_scores_rating`.

```
{ $match:  { city, date: { $in: [d1, d2] }, available: true } }
{ $group:  { _id: "$listing_id", a: { $sum: 1 } } }
{ $match:  { a: 2 } }
{ $lookup: { from: "listings", … } }
{ $sort:   { review_scores_rating: -1 } }
```

Real `explain('executionStats')` from the full-data run.

stage	metric
index used	`calendar { city, date, available }`
keys / docs examined	**3,012 / 3,012**
rows returned (pre-limit)	**1,826**
total time	**112 ms** over 1.6M Portland calendar rows

Q2 · Empty neighborhoods in a month

demo | indexed anti-join

Goal. Per (city, month), which neighborhoods had **zero** listings available.

1. `$match` calendar by city + date range + `available:true` → `$group` by `listing_id`
2. `$lookup` listings → project `neighborhood` → `$group` distinct → **active set**
3. App-side **anti-join** vs full `neighborhoods` collection for that city

Result excerpt (out/q2_results.json)

> **2 of 95** Portland neighborhoods were **dark** in 2026-06.

Indexes hit. `calendar { city, date, available }` · `listings { city, listing_id }` · `neighborhoods { city, neighborhood }`

> Anti-join in app code (set difference) — the active set is small enough to diff in-process; a `$lookup`-based set difference would scan the full nbhd collection per active row.

Q3 · Salem entire-home windows

demo | app-side scanner | min-nights aware

Goal. For each Salem entire-home listing, find every contiguous bookable window in a given month, **respecting per-day `minimum_nights`**.

1. **One** indexed `find` for candidate Salem entire-home listings **(uses `listings { city, neighborhood, room_type }`)**
2. **One** sorted `find` over `calendar { city, listing_id, date }` for the month
3. App-side: walk consecutive `available:true` runs; keep day **d** only if **remaining-run-length ≥ `minimum_nights[d]`**

Sample card — `Listing #234567 → 2025-12-04 → 2025-12-09 · 5 nights · min 2`

> Why app-side? A listing might say **available** on Saturday but require a 7-night minimum. If only Sat–Sun are open you can't actually book it. Mongo can't express this in aggregation alone.

Q4 · Portland trend, March → August

demo | reuses Q3

Goal. Total bookable nights per month for Portland entire-home listings, Mar → Aug.
Implementation. Q3's interval scanner in a 6-iteration loop.

> Deliberately **not** one giant aggregation. Each month is one indexed scan + the same scanner. Easy to add more months or filter by neighborhood.

Q5 · December reviews per (city, year)

demo | COLLSCAN — by design | 1,775 ms

```
{ $match: { date: { $regex: /^\d{4}-12-\d{2}$/ } } }
{ $group: { _id: { city: "$city",
                   year: { $substr: ["$date", 0, 4] } },
            review_count: { $sum: 1 } } }
{ $sort:  { city: 1, year: 1 } }
```

metric	value
reviews scanned	**3,200,722**
December matches	**193,340**
(city, year) groups returned	**62**
total time	**1,775 ms**

Date-as-string paid off. December extraction is a regex + `$substr`, no `$toDate` cost. If reviews grew to 100M+, add `reviews { date: 1 }`.

Q6 · Re-book reminder + same-host listings

demo · touches all 4 collections

Goal. For repeat (reviewer, listing) pairs: was the listing available in the same month as their previous review? What other listings does that host have in the same city?

1. Aggregate `reviews` → repeat (city, listing, reviewer) groups
2. **Per group:** `findOne` listing details · `findOne` calendar (`available:true`) · `find` other host listings
3. Emit enriched record

> **No new indexes needed** for any of the 6 queries — every per-record read in step 2 lands on an index already designed in Stage 2 for some other query. Strong signal the index design generalises.

> One aggregation + a few indexed point reads beats a single deeply-nested `$lookup` for both clarity **and** memory.

Critique — what we'd change

1. `host_id_1_city_1` — wrong key order

Should be `{ city, host_id }` so it shares a `city` prefix with our other three `listings` indexes. Affects Q6's same-host lookup.

2. Date-as-string call was right — but for the wrong reason

Stage 2 justified it on storage. The actual win was **query simplicity** (Q5's regex + `$substr`). Update the rationale.

3. Embed neighborhoods inside city documents

Eliminates the `$lookup` in Q2 entirely. Per-city array of canonical names is a few hundred bytes.

> What we **did** change between Stage 2 and Stage 3: **nothing** in the schema or indexes. Stage 3 is a true validation of Stage 2, not a redesign.

Lessons learned

About MongoDB

- `mongodb-memory-server` shrinks the design loop from minutes to seconds — make it the default for prototyping
- `explain('executionStats')` is the **single most valuable** design-validation tool — commit its output alongside query code
- Schema design happens at **index-design** time, not collection-design time

About cloud data systems generally

- Multi-step orchestration in app code beats nested `$lookup` for clarity **and** memory (Q3, Q4, Q6)
- "Use the planner" — pseudocode is guessing, `explain` is proof
- Ship the loader and the queries together; the schema isn't real until something has actually loaded into it

Thank you

Repo

`github.com/abachu2005/cs498-airbnb-mongodb-prototype`

Run it

```
cd stage3-prototype
npm install
npm start            # CLI evidence pipeline
npm run web          # interactive demo UI
```

Questions?

Evidence in `out/`

- `load_evidence.txt`
- `q1_results.json` … `q6_results.json`
- `explain_q1.json` … `explain_q6.json`
- `summary.json`

Companion docs

- `STAGE_3_Report_MongoDB_Airbnb.pdf`
- `STAGE_3_Code.pdf`